

C#

DO JEITO CERTO

AULA 10

Construindo Domínios Ricos



CAPÍTULO 1

O QUE É UM CÓDIGO LEGADO ?



Código legado

Projeto herdado

**Projeto sem cobertura
de testes**

**Qualquer sistema em
produção**

**Qualquer código escrito
há mais de um mês**

Projeto herdado

- E se o **código herdado** for de **boa qualidade**?
- Códigos **escritos por nós mesmos**, há anos atrás, também podem ser **confusos** e **desprovidos de boas práticas**

Projeto sem cobertura de testes

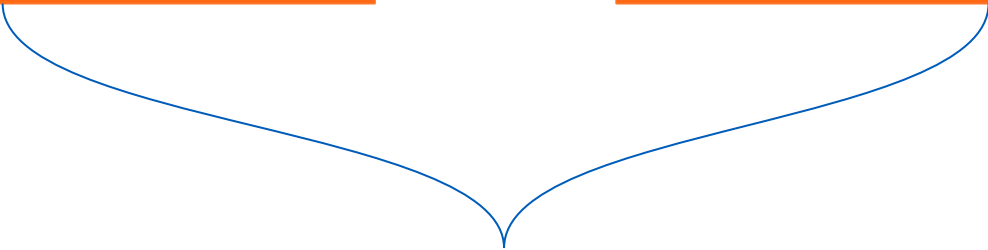
- Os **testes** podem ser **ruins**
- **Testes** que são **assertion-free** dão uma falsa sensação de segurança

Código legado não é apenas código antigo ou mal escrito — é código que já não compreendemos totalmente. Pode até ter testes e boa estrutura, mas sem clareza, confiança e contexto, qualquer sistema se torna difícil e arriscado de evoluir.



**Qualquer sistema em
produção**

**Qualquer código escrito
há mais de um mês**



Não dizem nada sobre o código em si, portanto não podem ser utilizadas para determinar se um código é legado

Código que requer um esforço significativo para ser mantido.

Um sistema se torna legado quando a nossa capacidade de pagar dívidas técnicas é menor do que a necessidade de contrair novas.



**Código
legado**

=

**Código
ruim**

Código legado

- Isso ocorre por conta do **software entropy**, ou seja, o quão **desorganizado** é um sistema
- Tomamos **atalhos**
- Aplicamos **soluções rápidas** para uma **entrega mais ágil**
- Acarreta, a médio/longo prazo, um **efeito dominó** de problemas
- Conforme esse código for crescendo, a **entropia** cresce também
- Eventualmente, tal sistema se torna um **big ball of mud**
- O que fazer com toda essa bagunça?



Reescrever tudo!

O código legado deve ser visto sob a perspectiva de negócio

Código é o
PASSIVO

Funcionalidade
é o **ATIVO**

$$\text{Sucesso} = \frac{\text{Funcionalidade}}{\text{Código}}$$

$$\frac{\text{Funcionalidade}}{\text{Código}} = \frac{\text{Nenhuma}}{\text{Muito}} = \text{Nenhum benefício}$$

Sem aumentar os ativos

Aumento do passivo

**Refatorar o código
legado**

+

**Entregas de valor para o
negócio**

” Quando a reescrita é algo aceitável? ”



Projeto não rentável



Projeto Pequeno

Manifesto para a modernização do legado

Resultados de Negócio

Acima de

Resultados Tecnológicos

Abordagem Iterativa e Entregas Parciais

Acima de

Projetos de Longa Duração

Trabalho por Componentes e Capabilities

Acima de

Migração de Aplicações Inteiras

Responsabilidade Federada

Acima de

Controle Centralizado

Escalabilidade e Resiliência

Acima de

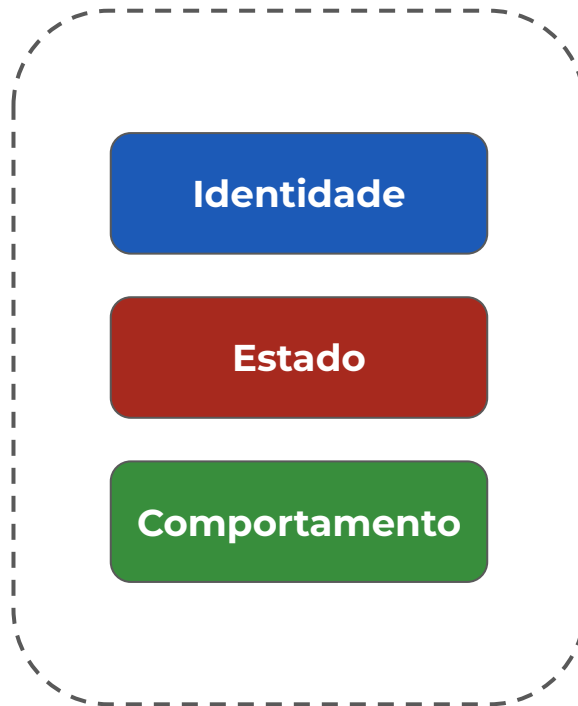
Soluções Temporárias



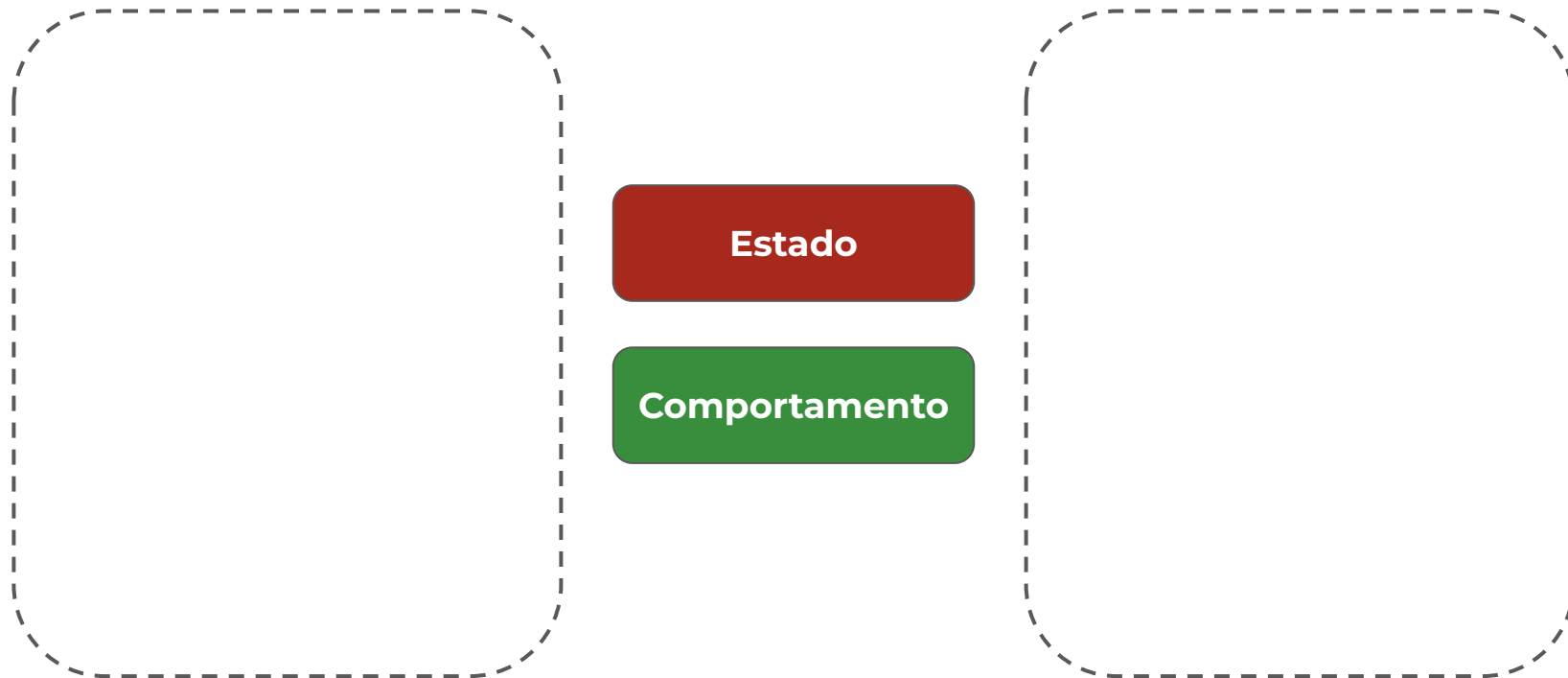
CAPÍTULO 2

MODELOS ANÊMICOS





O que é um modelo anêmico ?



O que é um modelo anêmico ?



Getters e setters públicos



Stateless services

Problemas na adoção de modelos anêmicos

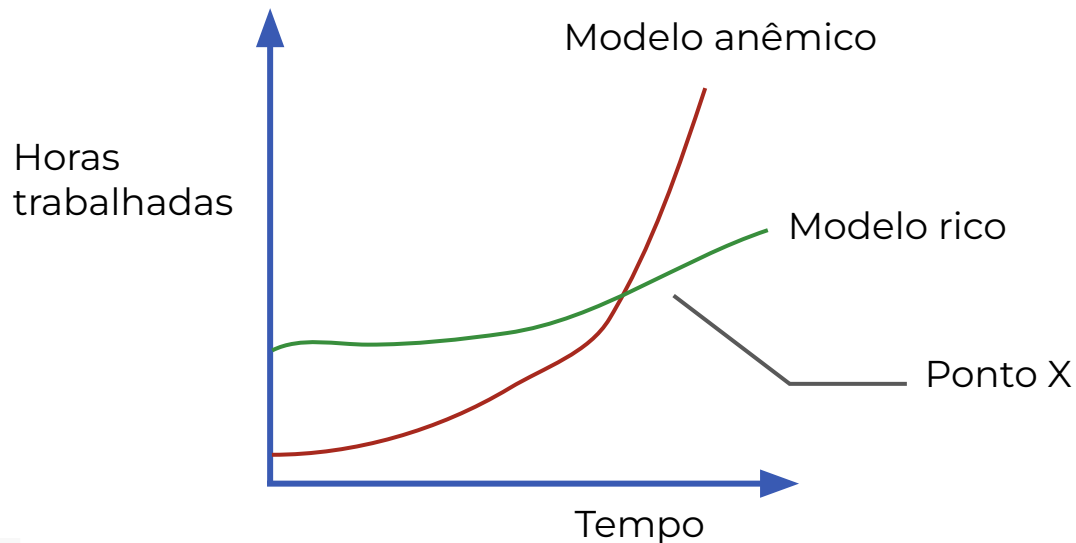
- Não cumpre as ideias de um design orientado a objetos.
- Em projetos grandes há uma tendência à duplicação de código.
- Dificuldade em encontrar os comportamentos desejados.
- Falta de encapsulamento.



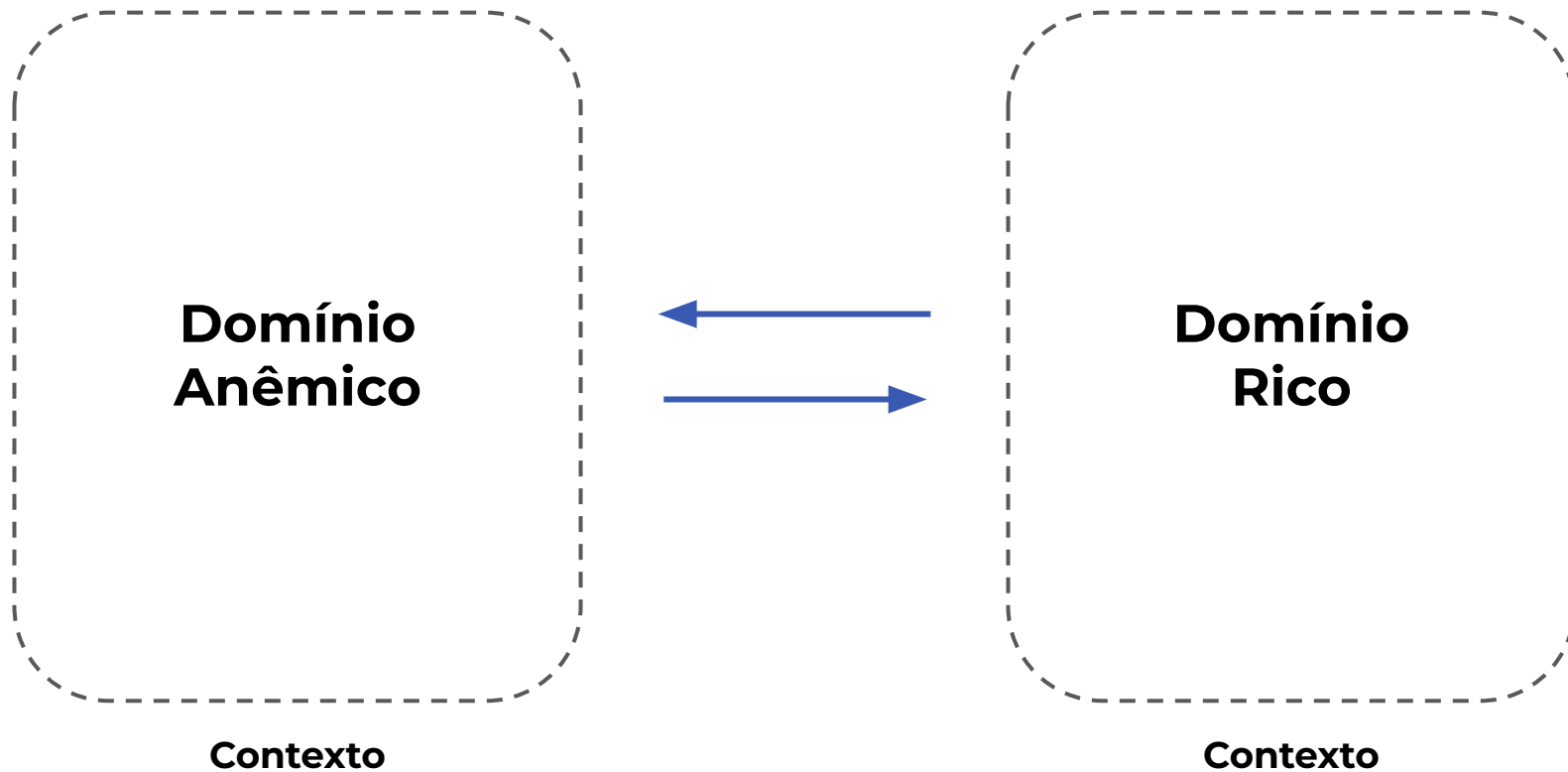
Vantagens na adoção de modelos anêmicos

- O código é intuitivo
- Fácil para ser implementado (ao menos no início)

Vantagens na adoção de modelos anêmicos



Relacionamento entre domínios



```
public class AnemicSquare
{
    public decimal SideLength { get; set; }
}

public class Services
{
    public static decimal CalculateArea(AnemicSquare square)
        => square.SideLength * square.SideLength;
}
```

CAPÍTULO 3

GUIA DE REFATORAÇÃO

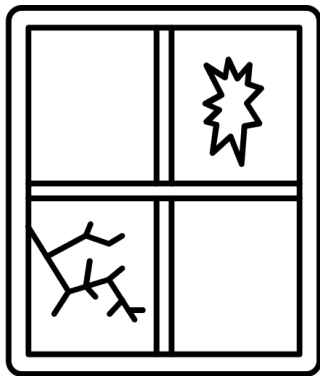


- Não refatorar um código que não está relacionado a nova funcionalidade
- A não ser que a refatoração seja o produto final



- Refatore de forma **incremental**, sem sacrificar a qualidade do código
- Não se preocupe em entender como toda a aplicação funciona para apenas então aplicar refatorações
- **Nem todo código** deve ser refatorado
- **Sempre** haverá código ruim





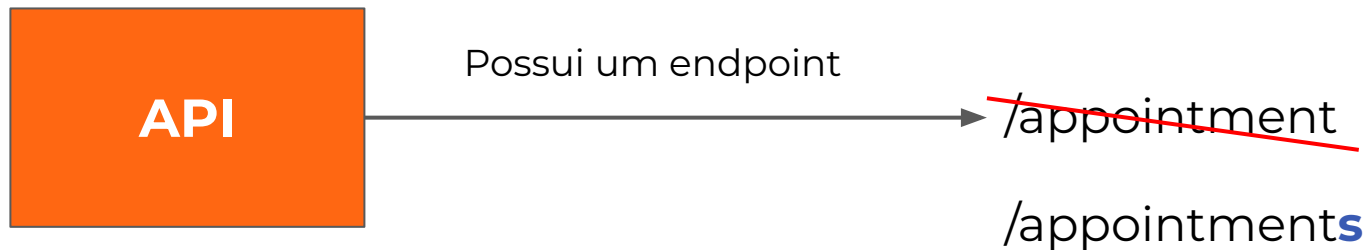
Cuidado com o efeito ***broken window***

Ao **aceitarmos** a primeira vez um **design ruim**, somos **mais propensos** a realizar novas **concessões**

CAPÍTULO 4

DESACOPLANDO O DOMÍNIO DOS CONTRATOS EXTERNOS





Breaking change

Desacoplando o domínio dos contratos externos

```
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
    public DateOnly DateOfBirth { get; set; }
}
```

Classe do domínio

Retorno direto da
classe de domínio

```
[HttpGet("All")]
public IActionResult ListPeople()
{
    return Ok(_peopleRepository.GetAll());
}
```

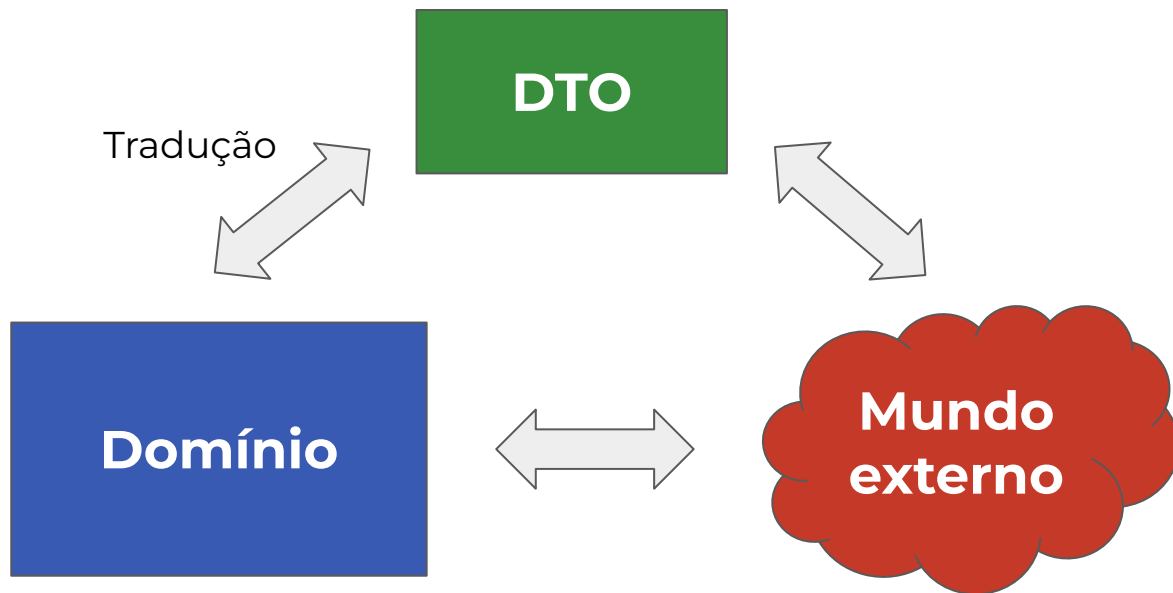
Desacoplando o domínio dos contratos externos

```
{  
  "Id": 125,  
  "Name": "Francisco Leonardo Schneider",  
  "DateOfBirth": "1996-11-11"  
}
```

FullName

Breaking change

- **DTO** é o acrônimo para **Data Transfer Object**



DTO

InputModel/Request

ViewModel/Response

Desacoplando o domínio dos contratos externos

```
public record PersonViewModel(  
    int Id,  
    string FullName,  
    DateOnly DateOfBirth);
```

DTO

Retorno do DTO

```
[HttpGet("All/v2")]  
public IActionResult ListPeopleV2()  
{  
    var people = _peopleRepository.GetAll();  
    return Ok(people  
        .Select(p => new PersonViewModel(  
            p.Id,  
            p.Name,  
            p.DateOfBirth)));  
}
```

CAPÍTULO 5

ENCAPSULAMENTO COMO TÉCNICA DE ENRIQUECIMENTO DO DOMÍNIO



”Operação de **finalização de um **atendimento**
veterinario online.”**

- Apenas o **próprio veterinário** pode **finalizar** o seu atendimento
- É **necessário** que tenha ocorrido o encontro na sala de vídeo entre o veterinário e o dono do pet
- Se o atendimento já estiver **finalizado**, deve **retornar sucesso**
- O atendimento não pode estar **cancelado**



Encapsulamento como técnica de enriquecimento do domínio

```
public void Finalize(int appointmentId, int veterinarianId)
{
    var appointment = _appointmentRepository.GetById(appointmentId);
    if (appointment is null)
        throw new Exception("Agendamento não encontrado.");

    if (!_appointmentRepository.HasMatch(appointmentId))
        throw new Exception("Não é possível finalizar o atendimento quando uma das partes não compareceu.");

    if (appointment.VeterinarianId != veterinarianId)
        throw new Exception("O atendimento não pode ser finalizado por este usuário");

    if (appointment.Status == EAppointmentStatus.Finalized)
        return;

    if (appointment.Status == EAppointmentStatus.Canceled)
        throw new Exception("O atendimento foi cancelado.");

    appointment.Status = EAppointmentStatus.Finalized;
    appointment.FinalizationDate = DateTime.UtcNow;

    _appointmentRepository.Update(appointment);
    _notificationService.NotifyAppointmentFinalization(appointment);
}
```

Encapsulamento como técnica de enriquecimento do domínio

```
public void Finalize(int appointmentId, int veterinarianId)
{
```

```
    var appointment = _appointmentRepository.GetById(appointmentId);
    if (appointment is null)
        throw new Exception("Agendamento não encontrado.");
```

```
    if (!_appointmentRepository.HasMatch(appointmentId))
        throw new Exception("Não é possível finalizar o atendimento quando uma das partes não compareceu.");
```

```
    if (appointment.VeterinarianId != veterinarianId)
        throw new Exception("O atendimento não pode ser finalizado por este usuário");
```

```
    if (appointment.Status == EAppointmentStatus.Finalized)
        return;
```

```
    if (appointment.Status == EAppointmentStatus.Canceled)
        throw new Exception("O atendimento foi cancelado.");
```

```
    appointment.Status = EAppointmentStatus.Finalized;
    appointment.FinalizationDate = DateTime.UtcNow;
```

```
    _appointmentRepository.Update(appointment);
    _notificationService.NotifyAppointmentFinalization(appointment);
```

```
}
```

Regras de negócios
esparsas

Encapsulamento como técnica de enriquecimento do domínio

```
public void Finalize(int appointmentId, int veterinarianId)
{
    var appointment = _appointmentRepository.GetById(appointmentId);
    if (appointment is null)
        throw new Exception("Agendamento não encontrado.");

    if (!_appointmentRepository.HasMatch(appointmentId))
        throw new Exception("Não é possível finalizar o atendimento quando uma das partes não compareceu.");

    if (appointment.VeterinarianId != veterinarianId)
        throw new Exception("O atendimento não pode ser finalizado por este usuário");

    if (appointment.Status == EAppointmentStatus.Finalized)
        return;

    if (appointment.Status == EAppointmentStatus.Canceled)
        throw new Exception("O atendimento foi cancelado.");

    appointment.Status = EAppointmentStatus.Finalized;
    appointment.FinalizationDate = DateTime.UtcNow;

    _appointmentRepository.Update(appointment);
    _notificationService.NotifyAppointmentFinalization(appointment);
}
```

Set de múltiplas
propriedades

Encapsulamento como técnica de enriquecimento do domínio

```
public void Finalize(int appointmentId, int veterinarianId)
{
    var appointment = _appointmentRepository.GetById(appointmentId);
    if (appointment is null)
        throw new Exception("Agendamento não encontrado.");

    if (!_appointmentRepository.HasMatch(appointmentId))
        throw new Exception("Não é possível finalizar o atendimento quando uma das partes não compareceu.");

    if (appointment.VeterinarianId != veterinarianId)
        throw new Exception("O atendimento não pode ser finalizado por este usuário");

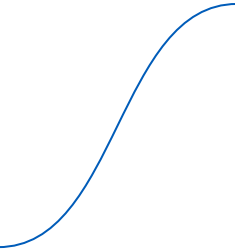
    if (appointment.Status == EAppointmentStatus.Finalized)
        return;

    if (appointment.Status == EAppointmentStatus.Canceled)
        throw new Exception("O atendimento foi cancelado.");

    appointment.Status = EAppointmentStatus.Finalized;
    appointment.FinalizationDate = DateTime.UtcNow;

    _appointmentRepository.Update(appointment);
    _notificationService.NotifyAppointmentFinalization(appointment);
}
```

Realização de ação
pós finalização



```
public class Appointment
{
    public int Id { get; set; }
    public int VeterinarianId { get; set; }
    public EAppointmentStatus Status { get; set; }
    public DateTime? FinalizationDate { get; set; }
}
```

Encapsulamento como técnica de enriquecimento do domínio

```
public class Appointment
{
    public Appointment(int id, int veterinarianId, EAppointmentStatus status, DateTime? finalizationDate)
    {
        Id = id;
        VeterinarianId = veterinarianId;
        Status = status;
        FinalizationDate = finalizationDate;
    }

    public int Id { get; }
    public int VeterinarianId { get; }
    public EAppointmentStatus Status { get; private set; }
    public DateTime? FinalizationDate { get; private set; }

    public Result Finalize(int veterinarianId)
    {
        if (Status == EAppointmentStatus.Finalized)
            return Result.Success();

        if (Status == EAppointmentStatus.Canceled)
            return Result.Failure("O atendimento foi cancelado.");

        if (VeterinarianId != veterinarianId)
            return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

        Status = EAppointmentStatus.Finalized;
        FinalizationDate = DateTime.UtcNow;
        return Result.Success();
    }
}
```

Encapsulamento como técnica de enriquecimento do domínio

```
public class Appointment
{
    public Appointment(int id, int veterinarianId, EAppointmentStatus status, DateTime? finalizationDate)
    {
        Id = id;
        VeterinarianId = veterinarianId;
        Status = status;
        FinalizationDate = finalizationDate;
    }

    public int Id { get; }
    public int VeterinarianId { get; }
    public EAppointmentStatus Status { get; private set; }
    public DateTime? FinalizationDate { get; private set; }

    public Result Finalize(int veterinarianId)
    {
        if (Status == EAppointmentStatus.Finalized)
            return Result.Success();

        if (Status == EAppointmentStatus.Canceled)
            return Result.Failure("O atendimento foi cancelado.");

        if (VeterinarianId != veterinarianId)
            return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

        Status = EAppointmentStatus.Finalized;
        FinalizationDate = DateTime.UtcNow;
        return Result.Success();
    }
}
```

Definição de construtor para criação da entidade num estado válido

Encapsulamento como técnica de enriquecimento do domínio

```
public class Appointment
{
    public Appointment(int id, int veterinarianId, EAppointmentStatus status, DateTime? finalizationDate)
    {
        Id = id;
        VeterinarianId = veterinarianId;
        Status = status;
        FinalizationDate = finalizationDate;
    }
}
```

```
public int Id { get; }
public int VeterinarianId { get; }
public EAppointmentStatus Status { get; private set; }
public DateTime? FinalizationDate { get; private set; }
```

```
public Result Finalize(int veterinarianId)
{
    if (Status == EAppointmentStatus.Finalized)
        return Result.Success();

    if (Status == EAppointmentStatus.Canceled)
        return Result.Failure("O atendimento foi cancelado.");

    if (VeterinarianId != veterinarianId)
        return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

    Status = EAppointmentStatus.Finalized;
    FinalizationDate = DateTime.UtcNow;
    return Result.Success();
}
```

Fechamento das
propriedades

Encapsulamento como técnica de enriquecimento do domínio

```
public class Appointment
{
    public Appointment(int id, int veterinarianId, EAppointmentStatus status, DateTime? finalizationDate)
    {
        Id = id;
        VeterinarianId = veterinarianId;
        Status = status;
        FinalizationDate = finalizationDate;
    }

    public int Id { get; }
    public int VeterinarianId { get; }
    public EAppointmentStatus Status { get; private set; }
    public DateTime? FinalizationDate { get; private set; }
```

Comportamento para finalização do atendimento

```
public Result Finalize(int veterinarianId)
{
    if (Status == EAppointmentStatus.Finalized)
        return Result.Success();

    if (Status == EAppointmentStatus.Canceled)
        return Result.Failure("O atendimento foi cancelado.");

    if (VeterinarianId != veterinarianId)
        return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

    Status = EAppointmentStatus.Finalized;
    FinalizationDate = DateTime.UtcNow;
    return Result.Success();
}
```

Encapsulamento como técnica de enriquecimento do domínio

```
public Result Finalize(int appointmentId, int veterinarianId)
{
    var appointment = _appointmentRepository.GetById(appointmentId);
    if (appointment is null)
        return Result.Failure("Agendamento não encontrado.");

    if (!_appointmentRepository.HasMatch(appointmentId))
        return Result.Failure("Não é possível finalizar o atendimento quando uma das partes não compareceu.");

    var result = appointment.Finalize(veterinarianId);
    if (result.IsFailure)
        return result;

    _appointmentRepository.Update(appointment);
    _notificationService.NotifyAppointmentFinalization(appointment);
    return Result.Success();
}
```

Encapsulamento como técnica de enriquecimento do domínio

Classe **Appointment** foi **enriquecida**
com **comportamento**

```
public Result Finalize(int appointmentId, int veterinarianId)
{
    var appointment = _appointmentRepository.GetById(appointmentId);
    if (appointment is null)
        return Result.Failure("Agendamento não encontrado.");

    if (!_appointmentRepository.HasMatch(appointmentId))
        return Result.Failure("Não é possível finalizar o atendimento quando uma das partes não compareceu.");

    var result = appointment.Finalize(veterinarianId);
    if (result.IsFailure)
        return result;

    _appointmentRepository.Update(appointment);
    _notificationService.NotifyAppointmentFinalization(appointment);
    return Result.Success();
}
```

Encapsulada regra de negócio para
finalização do atendimento

Encapsulamento como técnica de enriquecimento do domínio

A sua **resolução depende** de
uma **dependência externa**

```
public Result Finalize(int appointmentId, int veterinarianId)
{
```

```
    var appointment = _appointmentRepository.GetById(appointmentId);
    if (appointment is null)
        return Result.Failure("Agendamento não encontrado.");
```

```
    if (!_appointmentRepository.HasMatch(appointmentId))
        return Result.Failure("Não é possível finalizar o atendimento quando uma das partes não compareceu.");
```

```
    var result = appointment.Finalize(veterinarianId);
    if (result.IsFailure)
        return result;
```

```
    _appointmentRepository.Update(appointment);
    _notificationService.NotifyAppointmentFinalization(appointment);
    return Result.Success();
}
```

Regra de negócio intencionalmente
não tratada dentro da entidade de
domínio

Encapsulamento como técnica de enriquecimento do domínio

Nesses casos, é **recomendável** a **utilização** de **eventos de domínio**

Não é uma boa prática empilhá-las numa *Service* após a realização da operação

```
public Result Finalize(int appointmentId, int veterinarianId)
{
    var appointment = _appointmentRepository.GetById(appointmentId);
    if (appointment is null)
        return Result.Failure("Agendamento não encontrado.");

    if (!_appointmentRepository.HasMatch(appointmentId))
        return Result.Failure("Não é possível finalizar o atendimento quando uma das partes não compareceu.");

    var result = appointment.Finalize(veterinarianId);
    if (result.IsFailure)
        return result;

    _appointmentRepository.Update(appointment);
    _notificationService.NotifyAppointmentFinalization(appointment);
    return Result.Success();
}
```

A blue line starts from the text 'Ações pós operação são comuns' at the bottom right, curves upwards and to the left, and ends pointing directly at the line of code '_notificationService.NotifyAppointmentFinalization(appointment);' which is highlighted with a blue rectangular box.

Ações pós operação são comuns

CAPÍTULO 6

COMO TRABALHAR COM EVENTOS DE DOMÍNIO?



- Acontecimento que ocorreu no passado
- Evento que ocorreu **dentro do domínio** e que é propagado dentro do mesmo domínio
- Funciona em **memória**
- Frequentemente implementado com a biblioteca MediatR



Abordagem 1 - Publicação dos eventos de forma imediata

```
public record AppointmentFinalizedEvent(int Id) : INotification;
```

Implementação da interface **INotification**
do **MediatR**

Abordagem 1 - Publicação dos eventos de forma imediata

```
public class AppointmentFinalizedEventHandler : INotificationHandler<AppointmentFinalizedEvent>
{
    private readonly INotificationService _notificationService;

    public AppointmentFinalizedEventHandler(INotificationService notificationService)
    {
        _notificationService = notificationService;
    }

    public async Task Handle(AppointmentFinalizedEvent notification, CancellationToken cancellationToken)
    {
        await _notificationService
            .NotifyAppointmentFinalizationAsync(notification.Id, cancellationToken)
            .ConfigureAwait(false);
    }
}
```

Abordagem 1 - Publicação dos eventos de forma imediata

Implementação da interface
INotificationHandler do **MediatR**

```
public class AppointmentFinalizedEventHandler : INotificationHandler<AppointmentFinalizedEvent>
{
    private readonly INotificationService _notificationService;

    public AppointmentFinalizedEventHandler(INotificationService notificationService)
    {
        _notificationService = notificationService;
    }

    public async Task Handle(AppointmentFinalizedEvent notification, CancellationToken cancellationToken)
    {
        await _notificationService
            .NotifyAppointmentFinalizationAsync(notification.Id, cancellationToken)
            .ConfigureAwait(false);
    }
}
```

Abordagem 1 - Publicação dos eventos de forma imediata

```
public class AppointmentFinalizedEventHandler : INotificationHandler<AppointmentFinalizedEvent>
{
    private readonly INotificationService _notificationService;

    public AppointmentFinalizedEventHandler(INotificationService notificationService)
    {
        _notificationService = notificationService;
    }
}
```

```
public async Task Handle(AppointmentFinalizedEvent notification, CancellationToken cancellationToken)
{
    await _notificationService
        .NotifyAppointmentFinalizationAsync(notification.Id, cancellationToken)
        .ConfigureAwait(false);
}
```

Execução de **ações pós evento** correspondente

Abordagem 1 - Publicação dos eventos de forma imediata

```
public async Task<Result> FinalizeAsync(int veterinarianId, CancellationToken cancellationToken = default)
{
    if (Status == EAppointmentStatus.Finalized)
        return Result.Success();

    if (Status == EAppointmentStatus.Canceled)
        return Result.Failure("O atendimento foi cancelado.");

    if (VeterinarianId != veterinarianId)
        return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

    Status = EAppointmentStatus.Finalized;
    FinalizationDate = DateTime.UtcNow;
    await DomainEvents.Raise(new AppointmentFinalizedEvent(Id), cancellationToken).ConfigureAwait(false);

    return Result.Success();
}
```

Abordagem 1 - Publicação dos eventos de forma imediata

```
public async Task<Result> FinalizeAsync(int veterinarianId, CancellationToken cancellationToken = default)
{
    if (Status == EAppointmentStatus.Finalized)
        return Result.Success();

    if (Status == EAppointmentStatus.Canceled)
        return Result.Failure("O atendimento foi cancelado.");

    if (VeterinarianId != veterinarianId)
        return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

    Status = EAppointmentStatus.Finalized;
    FinalizationDate = DateTime.UtcNow;
    await DomainEvents.Raise(new AppointmentFinalizedEvent(Id), cancellationToken).ConfigureAwait(false);

    return Result.Success();
}
```

Efetua a **publicação** do **evento de domínio**

Abordagem 1 - Publicação dos eventos de forma imediata

```
public async Task<Result> FinalizeAsync(int veterinarianId, CancellationToken cancellationToken = default)
{
    if (Status == EAppointmentStatus.Finalized)
        return Result.Success();

    if (Status == EAppointmentStatus.Canceled)
        return Result.Failure("O atendimento foi cancelado.");

    if (VeterinarianId != veterinarianId)
        return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

    Status = EAppointmentStatus.Finalized;
    FinalizationDate = DateTime.UtcNow;
    await DomainEvents.Raise(new AppointmentFinalizedEvent(Id), cancellationToken).ConfigureAwait(false);

    return Result.Success();
}
```

Método da **classe de domínio** precisa se tornar **assíncrono** para atender a **publicação imediata** do **evento de domínio**

Abordagem 1 - Publicação dos eventos de forma imediata

```
public static class DomainEvents
{
    public static Func<IMediator> Mediator { get; set; }

    public static async Task Raise<T>(T args, CancellationToken cancellationToken = default)
        where T : INotification
    {
        var mediator = Mediator.Invoke();
        await mediator.Publish(args, cancellationToken).ConfigureAwait(false);
    }
}
```

- A classe **DomainEvents** é estática, portanto publica imediatamente o evento
- Dificulta a realização de testes
- Publica o evento antes da persistência dos dados



Abordagem 2 - Publicação adiada dos eventos

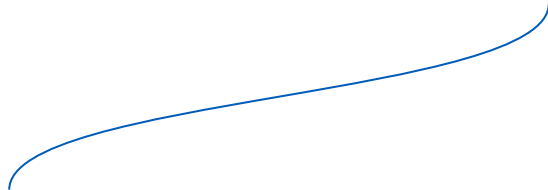


Interface **IDomainEvent** provê maior **expressividade**



```
public interface IDomainEvent : INotification { }
```

```
public record AppointmentFinalizedEvent(int Id) : IDomainEvent;
```



Evento passa a implementar a interface
IDomainEvent

Abordagem 2 - Publicação adiada dos eventos

```
public abstract class Entity<TKey> : IComparable<Entity<TKey>> where TKey : IComparable
{
    private List<IDomainEvent> _domainEvents;

    protected Entity(TKey id)
    {
        Id = id;
    }

    public virtual TKey Id { get; protected set; }

    [JsonIgnore]
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();

    protected void AddDomainEvent(IDomainEvent domainEvent)
    {
        _domainEvents ??= new List<IDomainEvent>();
        _domainEvents.Add(domainEvent);
    }

    // Rest of the class
}
```

Abordagem 2 - Publicação adiada dos eventos

```
public abstract class Entity<TKey> : IComparable<Entity<TKey>> where TKey : IComparable
{
```

```
    private List<IDomainEvent> _domainEvents;
```

```
    protected Entity(TKey id)
```

```
    {
        Id = id;
    }
```

```
    public virtual TKey Id { get; protected set; }
```

```
    [JsonIgnore]
```

```
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();
```

```
    protected void AddDomainEvent(IDomainEvent domainEvent)
```

```
    {
        _domainEvents ??= new List<IDomainEvent>();
        _domainEvents.Add(domainEvent);
    }
```

```
    // Rest of the class
```

Lista de **eventos de domínio**

Abordagem 2 - Publicação adiada dos eventos (EF)

```
public abstract class Entity<TKey> : IComparable<Entity<TKey>> where TKey : IComparable
{
    private List<IDomainEvent> _domainEvents;

    protected Entity(TKey id)
    {
        Id = id;
    }

    public virtual TKey Id { get; protected set; }

    [JsonIgnore]
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();

    protected void AddDomainEvent(IDomainEvent domainEvent)
    {
        _domainEvents ??= new List<IDomainEvent>();
        _domainEvents.Add(domainEvent);
    }

    // Rest of the class
}
```

Método para adicionar novos
eventos de domínio

Abordagem 2 - Publicação adiada dos eventos (EF)

```
public class Appointment : Entity<int>
{
    public Appointment(int id, int veterinarianId, EAppointmentStatus status, DateTime? finalizationDate)
        :base (id)
    {
        VeterinarianId = veterinarianId;
        Status = status;
        FinalizationDate = finalizationDate;
    }

    public int VeterinarianId { get; }
    public EAppointmentStatus Status { get; private set; }
    public DateTime? FinalizationDate { get; private set; }

    public Result Finalize(int veterinarianId)
    {
        if (Status == EAppointmentStatus.Finalized)
            return Result.Success();

        if (Status == EAppointmentStatus.Canceled)
            return Result.Failure("O atendimento foi cancelado.");

        if (VeterinarianId != veterinarianId)
            return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

        Status = EAppointmentStatus.Finalized;
        FinalizationDate = DateTime.UtcNow;
        AddDomainEvent(new AppointmentFinalizedEvent(Id));

        return Result.Success();
    }
}
```

Abordagem 2 - Publicação adiada dos eventos (EF)

```
public class Appointment : Entity<int>
```

```
{  
    public Appointment(int id, int veterinarianId, EAppointmentStatus status, DateTime? finalizationDate)  
        :base (id)  
    {  
        VeterinarianId = veterinarianId;  
        Status = status;  
        FinalizationDate = finalizationDate;  
    }  
  
    public int VeterinarianId { get; }  
    public EAppointmentStatus Status { get; private set; }  
    public DateTime? FinalizationDate { get; private set; }  
  
    public Result Finalize(int veterinarianId)  
    {  
        if (Status == EAppointmentStatus.Finalized)  
            return Result.Success();  
  
        if (Status == EAppointmentStatus.Canceled)  
            return Result.Failure("O atendimento foi cancelado.");  
  
        if (VeterinarianId != veterinarianId)  
            return Result.Failure("O atendimento não pode ser finalizado por este usuário.");  
  
        Status = EAppointmentStatus.Finalized;  
        FinalizationDate = DateTime.UtcNow;  
        AddDomainEvent(new AppointmentFinalizedEvent(Id));  
  
        return Result.Success();  
    }  
}
```

Herda da classe **Entity<T>**

Abordagem 2 - Publicação adiada dos eventos (EF)

```
public class Appointment : Entity<int>
{
    public Appointment(int id, int veterinarianId, EAppointmentStatus status, DateTime? finalizationDate)
        :base (id)
    {
        VeterinarianId = veterinarianId;
        Status = status;
        FinalizationDate = finalizationDate;
    }

    public int VeterinarianId { get; }
    public EAppointmentStatus Status { get; private set; }
    public DateTime? FinalizationDate { get; private set; }

    public Result Finalize(int veterinarianId)
    {
        if (Status == EAppointmentStatus.Finalized)
            return Result.Success();

        if (Status == EAppointmentStatus.Canceled)
            return Result.Failure("O atendimento foi cancelado.");

        if (VeterinarianId != veterinarianId)
            return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

        Status = EAppointmentStatus.Finalized;
        FinalizationDate = DateTime.UtcNow;
        AddDomainEvent(new AppointmentFinalizedEvent(Id));

        return Result.Success();
    }
}
```

Método **não** é mais **assíncrono**



Abordagem 2 - Publicação adiada dos eventos (EF)

```
public class Appointment : Entity<int>
{
    public Appointment(int id, int veterinarianId, EAppointmentStatus status, DateTime? finalizationDate)
        :base (id)
    {
        VeterinarianId = veterinarianId;
        Status = status;
        FinalizationDate = finalizationDate;
    }

    public int VeterinarianId { get; }
    public EAppointmentStatus Status { get; private set; }
    public DateTime? FinalizationDate { get; private set; }

    public Result Finalize(int veterinarianId)
    {
        if (Status == EAppointmentStatus.Finalized)
            return Result.Success();

        if (Status == EAppointmentStatus.Canceled)
            return Result.Failure("O atendimento foi cancelado.");

        if (VeterinarianId != veterinarianId)
            return Result.Failure("O atendimento não pode ser finalizado por este usuário.");

        Status = EAppointmentStatus.Finalized;
        FinalizationDate = DateTime.UtcNow;
        AddDomainEvent(new AppointmentFinalizedEvent(Id));
        return Result.Success();
    }
}
```

Adiciona o evento de domínio
para a **lista em memória**

Ainda **não publicou o evento**

Abordagem 2 - Publicação adiada dos eventos (EF)



```
public async Task<bool> SaveEntitiesAsync(CancellationTokentoken = default)
{
    var result = await base.SaveChangesAsync(cancellationTokentoken).ConfigureAwait(false);
    await _mediator.DispatchDomainEventsAsync(this, cancellationTokentoken).ConfigureAwait(false);
    return result > 0;
}
```

Abordagem 2 - Publicação adiada dos eventos (EF)

Persiste as alterações no **banco de dados**

```
public async Task<bool> SaveEntitiesAsync(CancellationToken cancellationToken = default)
{
    var result = await base.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    await _mediator.DispatchDomainEventsAsync(this, cancellationToken).ConfigureAwait(false);
    return result > 0;
}
```

Abordagem 2 - Publicação adiada dos eventos (EF)

```
public async Task<bool> SaveEntitiesAsync(CancellationToken cancellationToken = default)
{
    var result = await base.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    await _mediator.DispatchDomainEventsAsync(this, cancellationToken).ConfigureAwait(false);
    return result > 0;
}
```

Publica os **eventos de domínio** após
a persistência dos **dados**

Abordagem 2 - Publicação adiada dos eventos (EF)

```
public static class MediatorExtension
{
    internal static async Task DispatchDomainEventsAsync(
        this IMediator mediator,
        DbContext ctx,
        CancellationToken cancellationToken = default)
    {
        var domainEntities = ctx.ChangeTracker
            .Entries<IEntity>()
            .Where(x => x.Entity?.DomainEvents?.Any() == true);

        if (!domainEntities.Any())
            return;

        var domainEvents = domainEntities
            .SelectMany(x => x.Entity.DomainEvents)
            .ToList();

        foreach (var entity in domainEntities)
            entity.Entity.ClearDomainEvents();

        foreach (var domainEvent in domainEvents)
            await mediator.Publish(domainEvent, cancellationToken).ConfigureAwait(false);
    }
}
```

- Utilize eventos de domínio para tratar os efeitos colaterais de alterações em seu domínio
- O **MediatR** é uma boa opção para a publicação dos eventos de domínio
- Os eventos são publicados de forma **síncrona**
- Cuidado com a realização de operações muito custosas nos **event handlers**



- **Eventos de integração** servem para as notificar alterações realizadas no domínio de determinado sistema
- Utilize um evento de domínio para publicar um **evento de integração** dentro do mesmo sistema caso a operação a ser realizada for **custosa**
- Utilize eventos de integração como **forma de integração assíncrona** entre sistemas

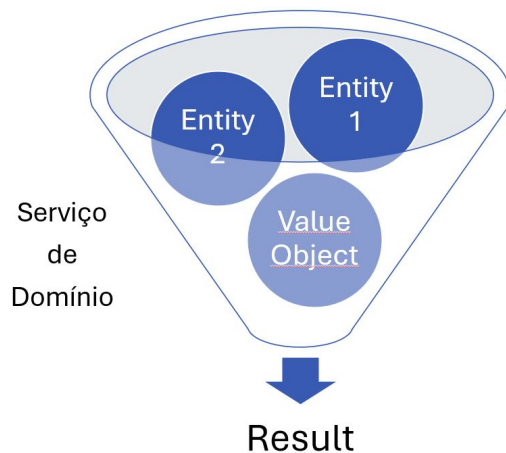


CAPÍTULO 7

QUANDO UTILIZAR SERVIÇOS DE DOMÍNIO?



- Quando **determinada** regra de negócio **não pertence exclusivamente** a uma classe do domínio
- Quando há a **necessidade** de integração com **recursos externos**



- Ambas são classes que **trabalham** com entidades e value objects
- A **principal diferença** entre um serviço de domínio e um serviço de aplicação é que o serviço de domínio **possui regra de negócio**, enquanto o de **aplicação não**



Quando utilizar o serviço de domínio ?



```
public void WithdrawMoney(decimal amount)
{
    _atm.DispenseMoney(amount);
    decimal amountWithCommission = _atm.CalculateAmountWithCommission(amount);

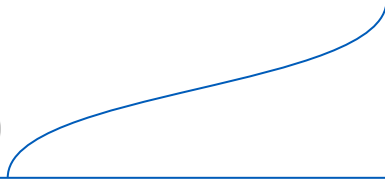
    _paymentGateway.ChargePayment(amountWithCommission);
    _repository.Save(_atm);
}
```

Quando utilizar o serviço de domínio ?



Utiliza a entidade **Atm** para **realizar** algumas **ações de negócio**

```
public void WithdrawMoney(decimal amount)
{
    _atm.DispenseMoney(amount);
    decimal amountWithCommission = _atm.CalculateAmountWithCommission(amount);
    _paymentGateway.ChargePayment(amountWithCommission);
    _repository.Save(_atm);
}
```



Quando utilizar o serviço de domínio ?



```
public void WithdrawMoney(decimal amount)
{
    _atm.DispenseMoney(amount);
    decimal amountWithCommission = _atm.CalculateAmountWithCommission(amount);

    _paymentGateway.ChargePayment(amountWithCommission);
    _repository.Save(_atm);
}
```



Torna essas **ações** realizadas no **domínio** visíveis ao **mundo externo** (banco de dados e serviço de pagamento)

Quando utilizar o serviço de domínio ?

```
public class AtmService // Domain service
{
    public decimal DispenseAndCalculateCommission(Atm atm, decimal amount)
    {
        atm.DispenseMoney(amount);
        return atm.CalculateAmountWithCommission(amount);
    }
}
```



```
public void WithdrawMoney(decimal amount)
{
    decimal amountWithCommission = _atmService.DispenseAndCalculateCommission(_atm, amount);

    _paymentGateway.ChargePayment(amountWithCommission);
    _repository.Save(_atm);
}
```

Quando utilizar o serviço de domínio ?

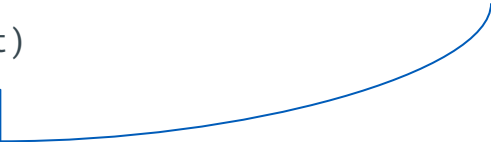


E se o cliente (nesse caso, o serviço de aplicação), **esquecer de realizar a validação?**

```
public void WithdrawMoney(decimal amount)
{
    if (!_atm.CanDispenseMoney(amount))
        return;

    _atm.DispenseMoney(amount);
    decimal amountWithCommission = _atm.CalculateAmountWithCommission(amount);

    _paymentGateway.ChargePayment(amountWithCommission);
    _repository.Save(_atm);
}
```



Quando utilizar o serviço de domínio ?



Cabe ao domínio manter sua **coesão**.

```
public void DispenseMoney(decimal amount)
{
    if (!CanDispenseMoney(amount))
        throw new InvalidOperationException();

    /* ... */
}
```



- Vamos assumir que o serviço de pagamento pode falhar por conta de saldo insuficiente, então não devemos permitir o saque de dinheiro



” Vamos assumir que o serviço de pagamento pode falhar por conta de saldo insuficiente, então não devemos permitir o saque de dinheiro ”

Quando trabalhar com serviços de domínio ?



```
public void WithdrawMoney(decimal amount)
{
    if (!_atm.CanDispenseMoney(amount))
        return;

    decimal amountWithCommission = _atm.CalculateAmountWithCommission(amount);
    Result result = _paymentGateway.ChargePayment(amountWithCommission);

    if (result.IsFailure)
        return;

    _atm.DispenseMoney(amount);
    _repository.Save(_atm);
}
```



Quando trabalhar com serviços de domínio ?

Esse **IF** é uma **regra de negócio**

Ele **decide quando** o **dinheiro** será **liberado**

```
public void WithdrawMoney(decimal amount)
{
    if (!_atm.CanDispenseMoney(amount))
        return;

    decimal amountWithCommission = _atm.CalculateAmountWithCommission(amount);
    Result result = _paymentGateway.ChargePayment(amountWithCommission);

    if (result.IsFailure)
        return;

    _atm.DispenseMoney(amount);
    _repository.Save(_atm);
}
```

Quem está tomando essa decisão **não é** a **entidade de domínio Atm**

E sim o **serviço de aplicação**

O que fazer nessa situação?

- Implementar um serviço de domínio
- Um serviço de domínio pode possuir regras de negócio que requerem informações do mundo externo

Quando trabalhar com serviços de domínio ?

```
public sealed class AtmService // Domain service
{
    private PaymentGatewayService _paymentGateway = new();

    public void WithdrawMoney(Atm atm, decimal amount)
    {
        if (!atm.CanDispenseMoney(amount))
            return;

        decimal amountWithCommission = atm.CalculateAmountWithCommission(amount);
        Result result = _paymentGateway.ChargePayment(amountWithCommission);

        if (result.IsFailure)
            return;

        atm.DispenseMoney(amount);
    }
}
```



Quando trabalhar com serviços de domínio ?



```
public void WithdrawMoney(decimal amount)
{
    Atm atm = _repository.Get();
    _atmService.WithdrawMoney(atm, amount);
    _repository.Save(_atm);
}
```

- Idealmente, **serviços de domínio** possuem regras de negócio, enquanto que os serviços de aplicação, não
- Por vezes é **OK** manter um pouco de **regra de negócio** que não encaixa em nenhuma entidade no serviço de aplicação, sem criar um serviço de domínio para todos os casos. Garanta apenas que:
 - Essa **lógica** não seja replicada em outros serviços de aplicação
 - Ela não seja muito complexa

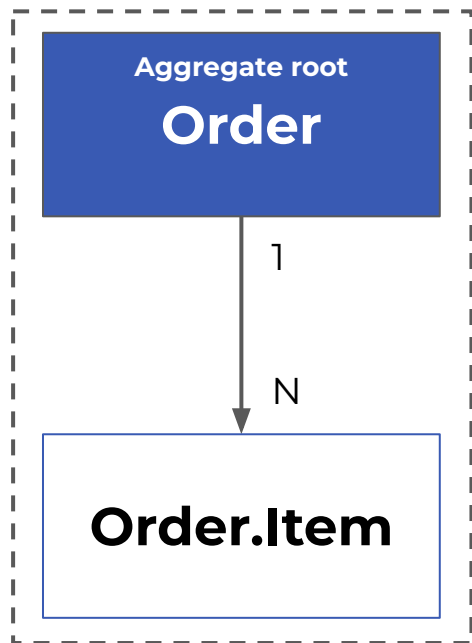
- **Utilize** serviços de domínio quando a regra de negócio **não pertence** a uma entidade específica
- Utilize serviços de domínio quando a lógica **não** pode ser **atribuída a entidade**, pois isso quebraria seu isolamento
- Um serviço de domínio pode ser injetado em uma entidade, se:
 - For puro, ou seja, isolado, não dependendo do mundo externo

CAPÍTULO 8

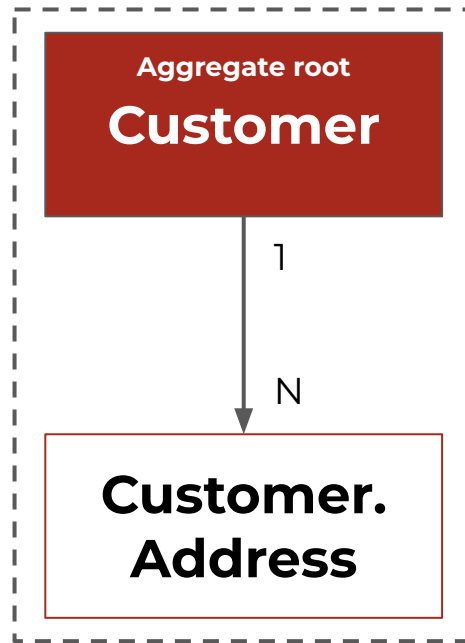
AGREGADOS NA PRÁTICA



Transação única



Transação única

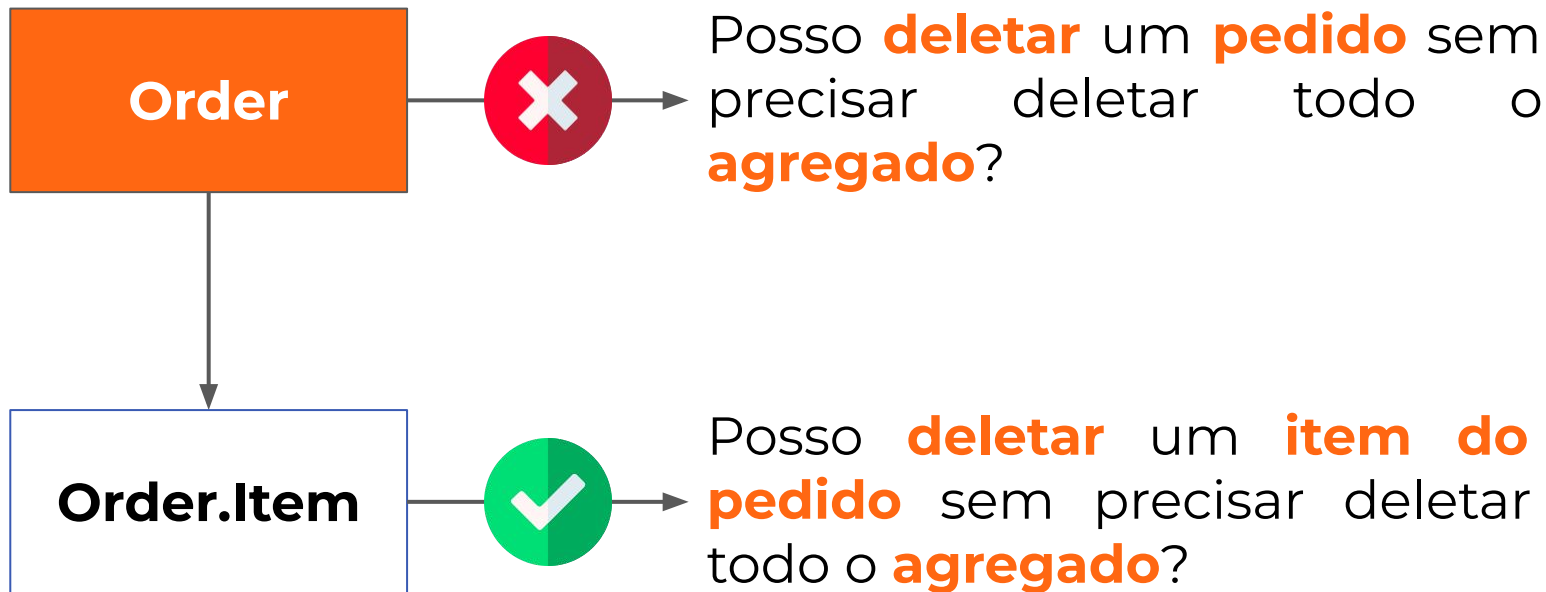


Como descobrir quem é o aggregate root ?



Descubra se ao **deletar o objeto**, o restante dos **objetos** precisam ser **deletados também** para que haja **coerência**

Como descobrir quem é o aggregate root ?



```
public interface IAggregateRoot
{
    IReadOnlyCollection<IDomainEvent> DomainEvents { get; }
    void AddDomainEvent(IDomainEvent domainEvent);
    void ClearDomainEvents();
}

public abstract class AggregateRoot<TKey> : Entity<TKey>, IAggregateRoot where TKey : IComparable
{
    private List<IDomainEvent> _domainEvents;

    protected AggregateRoot(TKey id) : base(id)
    {
        _domainEvents = new List<IDomainEvent>();
    }

    [JsonIgnore]
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _domainEvents.AsReadOnly();

    public void AddDomainEvent(IDomainEvent domainEvent)
    {
        _domainEvents ??= new List<IDomainEvent>();
        _domainEvents.Add(domainEvent);
    }

    public void ClearDomainEvents()
    {
        _domainEvents.Clear();
    }
}
```

```
public partial class Order : AggregateRoot<int>
{
    private List<Item> _items;

    public Order(int id, int customerId, DateTime createdAt, decimal amount, EOrderStatus status, List<Item> items)
        :base(id)
    {
        _items = items;
        CustomerId = customerId;
        CreatedAt = createdAt;
        Amount = amount;
        Status = status;
    }

    public int CustomerId { get; }
    public DateTime CreatedAt { get; }
    public decimal Amount { get; }
    public EOrderStatus Status { get; }
    public IReadOnlyCollection<Item> Items => _items.AsReadOnly();
}
```

O **root** herda da classe **AggregateRoot<>** e não mais da **Entity<>**

```
public partial class Order : AggregateRoot<int>
{
    private List<Item> _items;

    public Order(int id, int customerId, DateTime createdAt, decimal amount, EOrderStatus status, List<Item> items)
        :base(id)
    {
        _items = items;
        CustomerId = customerId;
        CreatedAt = createdAt;
        Amount = amount;
        Status = status;
    }

    public int CustomerId { get; }
    public DateTime CreatedAt { get; }
    public decimal Amount { get; }
    public EOrderStatus Status { get; }
    public IReadOnlyCollection<Item> Items => _items.AsReadOnly();
}
```

```
public partial class Order : AggregateRoot<int>
{
    private List<Item> _items;

    public Order(int id, int customerId, DateTime createdAt, decimal amount, EOrderStatus status, List<Item> items)
        :base(id)
    {
        _items = items;
        CustomerId = customerId;
        CreatedAt = createdAt;
        Amount = amount;
        Status = status;
    }

    public int CustomerId { get; }
    public DateTime CreatedAt { get; }
    public decimal Amount { get; }
    public EOrderStatus Status { get; }
    public IReadOnlyCollection<Item> Items => _items.AsReadOnly();
}
```

Lista de **itens**


```
public partial class Order
{
    public class Item : Entity<int>
    {
        public Item(int id, string description, decimal amount)
            : base(id)
        {
            Description = description;
            Amount = amount;
        }

        public string Description { get; }
        public decimal Amount { get; }
    }
}
```

Herda de **Entity<>**

```
public partial class Order
{
    public class Item : Entity<int>
    {
        public Item(int id, string description, decimal amount)
            : base(id)
        {
            Description = description;
            Amount = amount;
        }

        public string Description { get; }
        public decimal Amount { get; }
    }
}
```

```
public partial class Order  
{  
    public class Item : Entity<int>
```

```
{  
    public Item(int id, string description, decimal amount)  
        : base(id)  
    {  
        Description = description;  
        Amount = amount;  
    }  
  
    public string Description { get; }  
    public decimal Amount { get; }  
}  
}
```

A classe **Order** é **partial**,
pois a classe **Item** está
dentro dela

Não tem **como haver** um
item sem um **pedido**

```
public interface IOrderRepository
{
    Task InsertAsync(Order oder, CancellationToken cancellationToken = default);
    Task UpdateAsync(Order order, CancellationToken cancellationToken = default);
}
```

- Nem sempre teremos **agregados complexos**
- Agregados **devem** se comunicar apenas entre os root's
- **Customer** não conversa com **Order.Item**
- Muitas FK's para entidades filhas podem apresentar um problema de modelagem
- “**Agregados de um**“, ou seja, sem filhos, são aceitáveis
- Observar a regra de deleção em cascata para auxiliar na identificação do root

CAPÍTULO 9

BOAS PRÁTICAS

- **YAGNI** é o acrônimo para **You aren't going to need it**
- Requisitos de negócio mudam constantemente
- O código é um **passivo**, não um **ativo**



- **Expõe** apenas o necessário para alteração
- Não permite que o modelo de domínio **caia num estado inválido**
- **Não** é a mesma coisa que **isolamento**

Encapsulamento



Protege o que deve ser
imutável

Isolamento



Separação de
responsabilidades



Recomendações

Bibliografia

- Livro: Implementando Domain Driven Design, Vaughn Vernon
- Livro: Domain Driven Design Destilado, Vaughn Vernon

Artigos

- [Manifesto para Modernização de Sistemas Legados](#)
- [Obsessão por tipos primitivos](#)

Vídeos

- [DDD do jeito certo](#)

CLUBE DE ESTUDOS
BY ELEMAR JR 

ELEMAR